

CSE3016 - AWS SOLUTION ARCHITECT

Presented By

Dr Jasmine Selvakumari Jeya I

Senior Associate Professor Grade 2/SCOPE

VIT Bhopal University

MODULE II

Compute – Amazon EC2 (Elastic Compute Cloud), EC2 Instance Types, EC2 Pricing Models, Elastic IP, AWS Elastic Beanstalk, AWS Fargate, Amazon Elastic Container Service (ECS), Amazon Elastic Kubernetes Service (EKS).

Amazon EC2 (Elastic Compute Cloud)

- EC2 is an **on-demand virtual computing service** from AWS.
- Lets you **rent virtual computers (instances)** to run applications.
- **Pay only for what you use.**
- **Core concept of EC2:**
 - **Elastic:** Easily scale the number or size of instances.
 - **Compute:** Provides CPU, memory, and storage for workloads.
 - **Cloud:** Runs on AWS's global data center infrastructure.

Core Components of an EC2 Instance

- **Amazon Machine Image (AMI)**

- Pre-configured template with OS, patches, and software.
- Can use AWS-provided, community, marketplace, or custom AMIs.

- **Instance Types**

- Hardware profiles optimized for different tasks:
 - **t family** (t2.micro, t3.large): General purpose, web servers, small databases.
 - **m family** (m5.xlarge): Balanced CPU, memory, networking.
 - **c family** (c5.large): Compute optimized, CPU-intensive tasks.
 - **r family** (r5.large): Memory optimized, large databases, in-memory caches.

- **Elastic Block Store (EBS)**

- Persistent block-level storage, like a virtual hard drive.
- Data stays intact even if instance stops or terminates.

- **Security Groups**

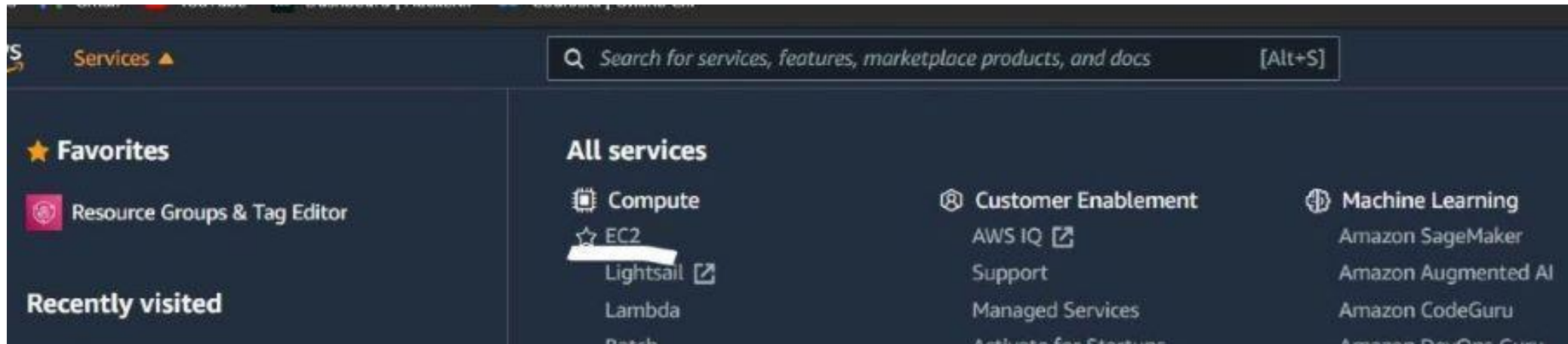
- Virtual firewall controlling inbound/outbound traffic.
- Rules define allowed protocols, ports, and IP addresses.

- **Key Pairs**

- Public-private key set for secure login to Linux instances via SSH (Secure Shell).

Amazon EC2 (Elastic Compute Cloud) Linux Instances

- **Step 1:** First login into your AWS account. Once you are directed to the management console. From the left click on "**Services**" and from the listed options click on **EC2**.



- **Step 2:** Afterward, you will be redirected to the EC2 console. Here is the image attached to refer to various features in EC2.

New EC2 Experience
Tell us what you think

EC2 Dashboard *New*

- Events
- Tags
- Limits

▼ **Instances**

- Instances** *New*
- Instance Types
- Launch Templates
- Spot Requests
- Savings Plans
- Reserved Instances *New*
- Dedicated Hosts
- Capacity Reservations

▼ **Images**

- AMIs

Welcome to the new EC2 console!
We're redesigning the EC2 console to make it easier to use and improve performance. We'll release new screens periodically. We encourage you to try them and let us know where we can make improvements. To switch between the old console and the new console, use the New EC2 Experience toggle.

Resources

You are using the following Amazon EC2 resources in the Asia Pacific (Mumbai) Region:

Instances (running)	0	Dedicated Hosts	0
Elastic IPs	0	Instances	1
Key pairs	1	Load balancers	0
Placement groups	0	Security groups	3
Snapshots	0	Volumes	0

Account attributes

Supported platforms

- VPC

Default VPC
vpc-7d4a8f16

Settings

- EBS encryption
- Zones
- Default credit specification
- Console experiments

Explore AWS

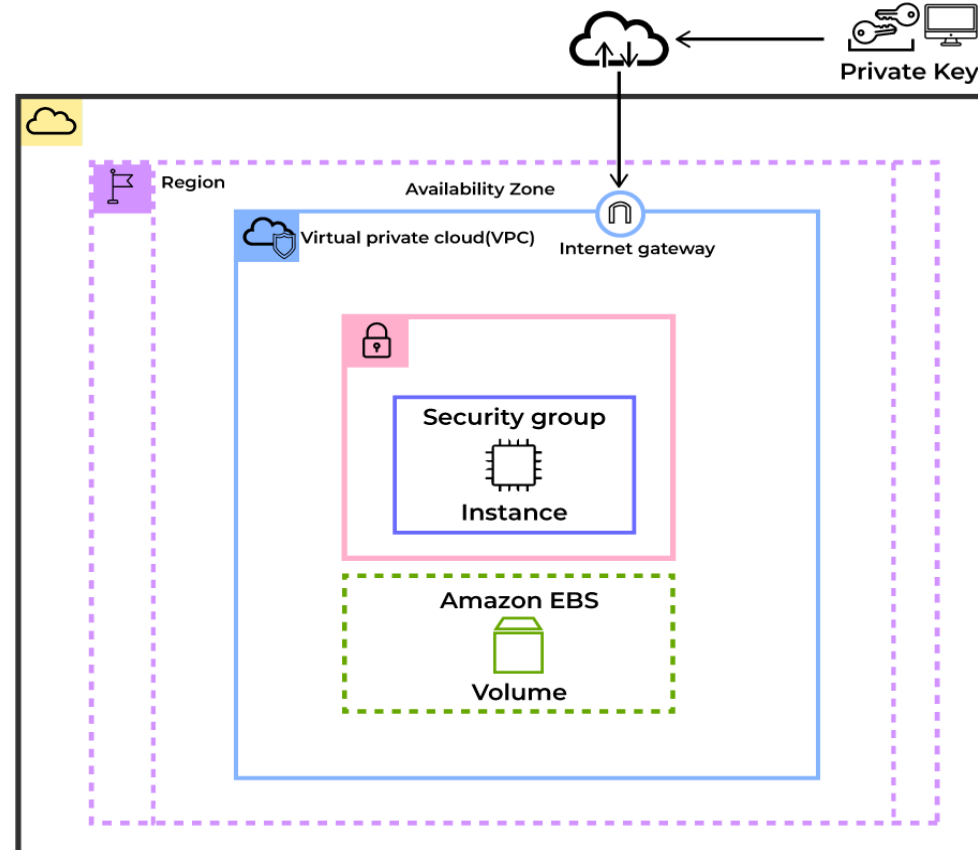
Easily size, configure, and deploy Microsoft SQL Server Always On availability groups on AWS using the AWS Launch Wizard for SQL Server. [Learn more](#)

Working of AWS EC2

Instead of buying and managing your own servers, EC2 gives you a virtual machine, where you can run websites, apps, or even big data tasks.

- Choose the memory, storage, and CPU you need, and stop the instance when done.
- EC2 offers secure, reliable, high-performance, and cost-effective infrastructure.
- Deploy applications without managing physical hardware.
- Secure your instance using VPC, Subnets, and Security Groups.
- Attach Auto Scaling to scale EC2 based on demand.
- Automatically scale up or down based on traffic.

- The following figure shows the EC2-Instance which is deployed in VPC (Virtual Private Cloud).



Features of AWS EC2 (Elastic Compute Cloud)

1. AWS EC2 Functionality

- EC2 provides a **virtual computing platform** to run operations and launch multiple instances.
- Enhances **security** and allows **full customization** of the virtual environment.
- Offers **default AMIs** with pre-configured OS and resources.
- Users can **create custom AMIs** to save preferred configurations for future use, avoiding repeated setup.

2. AWS EC2 Operating Systems

- Wide range of **operating systems** available when selecting an AMI.
- Users can also **upload their own OS** and use it while launching an instance.
- Commonly preferred OS on EC2 includes:
 - **Linux OS** (Amazon Linux, Ubuntu, SUSE, Red Hat)
 - **Windows Server**

Select an Operating System



- [Amazon Linux](#)
- Windows Server
- Ubuntu Server
- SUSE Linux
- Red Hat Linux

3. AWS EC2 Software

- EC2 offers a **wide variety of software options** for users.
- Software is available through the **AWS Marketplace**.
- Popular software includes **SAP, LAMP, Drupal**, and many more.

4. AWS EC2 Scalability and Reliability

- EC2 allows **scaling up or down** based on workload needs.
- Flexible **volumes and snapshots** enhance reliability.
- Highly **trusted by large organizations** (e.g., Amazon, Flipkart) for handling **high traffic** scenarios.

Amazon EC2 - Instance Types

- Virtual Machines (VMs) are the core compute units of cloud Infrastructure as a Service (IaaS).
- Cloud providers like **Amazon EC2**, **Azure Virtual Machines**, and **Google Compute Engine** let organizations rent VMs on demand.
- VMs provide **CPU, memory, and storage** without the need for physical hardware.
- A VM functions like a physical server: you can **connect, install an OS, deploy software, and run workloads**.
- VMs exist in the provider's **global cloud**, offering **flexibility, scalability, and automation** beyond traditional on-premises setups.

How to Read an EC2 Instance Type Name

- **Instance Type Names** (e.g., m6a.4xlarge) follow a logical pattern.
- **m – Instance Family:** Shows the primary purpose.
 - m = General Purpose
- **6 – Generation:** Indicates hardware generation. Higher number → newer & more powerful.
- **a – Processor Type:** Shows processor type.
 - a = AMD, i = Intel, g = AWS Graviton (ARM)
- **.4xlarge – Instance Size:** Determines vCPU, memory, and network resources.
- Sizes double at each step (large → xlarge → 2xlarge → 4xlarge).
- **Extra letters** may indicate special features:
 - d = local NVMe SSD storage
 - n = enhanced networking

What are EC2 Instance Types?

- **EC2 Instance Types** define the hardware of the host computer for your instance.
- Each type offers different **combinations of CPU, memory, storage, and networking**.
- This variety helps you choose the most suitable instance for your **application needs**:
 - **General-purpose computing**
 - **Memory-intensive processing**
 - **Accelerated computing with GPUs**

Why are Different Instance Types Important

- **Choose an EC2 instance based on your application's needs.**
- Different workloads need different resources:
 - **Web servers:** Balanced CPU and memory
 - **Big data analytics:** High memory and fast storage
 - **Machine learning:** GPUs for faster processing
- **AWS instance families** are optimized for specific use cases.
- Picking the right instance helps **improve performance** and **control costs**.

Categories of AWS EC2 Instance Types

- AWS groups EC2 instances into several families based on their target use cases. Here are the main categories:

Instance Category	Key Features	Use Cases	Examples
General Purpose Instances	Balanced CPU, memory, and networking resources	Web servers, small databases, development environments	T3, M5, M6g
Compute Optimized Instances	High CPU power compared to memory	Batch processing, gaming servers, machine learning inference	C5, C6g
Memory Optimized Instances	More memory compared to CPU	High-performance databases, big data analytics, real-time processing	R5, X1, Z1d
Storage Optimized Instances	Designed for workloads needing high, fast local storage	Data warehousing, big data, log processing	I3, D2, H1
Accelerated Computing Instances	Specialized hardware like GPUs or FPGAs for faster processing	Machine learning, graphics rendering, scientific simulations	P3, G4, F1

1. General Purpose Instances

- **General Purpose Instances** provide a balanced mix of CPU, memory, and networking.
- Suitable for applications that **don't need specialized hardware**.
- **Key Features:**
 - Balanced CPU, memory, and network performance
 - Versatile for many workloads
 - Cost-effective for common use cases

EC2 General-Purpose Instance Types:

- **M Series (M7g, M6i, M5):**
 - Balanced resources for small/medium databases, enterprise apps, web servers, dev/test environments
 - Newer generations (e.g., M7g with Graviton3) offer better price-performance
- **T Series (T4g, T3, T2):**
 - Burstable performance with baseline CPU + ability to burst
 - Ideal for apps with variable CPU needs: small web servers, microservices, dev environments, CI/CD pipelines
- **A1 Series:**
 - ARM-based with AWS Graviton processors
 - Good price-performance for scale-out workloads and ARM-compatible apps
- **Mac Series:**
 - Mac mini instances for **macOS development and testing**

Applications of EC2 General-Purpose Instances:

- **Web Servers:** Flexible and scalable platform for hosting web applications.
- **Development & Test Environments:** Cost-effective solution for building, testing, and deploying applications.
- **Content Delivery:** Can host CDNs to deliver content worldwide with low latency and high performance.
- **Summary:** General-purpose instances are versatile and scalable, suitable for many business applications.

2. Compute Optimized Instances:

- Designed for tasks that need **high processing power (CPU-heavy workloads)**.
- Ideal for **gaming servers, scientific modeling, machine learning, batch processing**.
- **Key Idea:** Use when software needs more **CPU** than memory or storage.
- **Key Features:**
 - Strong CPUs for fast computations
 - Balanced CPU, memory, and networking resources
 - Flexible for a variety of workloads
 - Cost-effective and scalable for growing needs

Use Cases:

- Web servers and application servers.
- Small to medium databases.
- Development and testing environments.
- Content management systems and backend servers.
- Business applications with moderate resource needs.

Families:

- **C Series (e.g., C7g, C6i, C5):** Excellent for batch processing workloads, high-performance web servers, scientific modeling, media transcoding, dedicated gaming servers, and machine learning inference. Graviton-powered C7g instances provide significant price-performance improvements.

3. Memory Optimized Instances

Memory Optimized Instances are designed to provide a large amount of RAM relative to CPU power. They are ideal for applications that require fast, efficient processing of large datasets stored in memory.

Key Features:

- High memory capacity with fast access
- Low latency memory performance
- Enhanced networking and storage support

Use Cases:

- High-performance databases (SQL, NoSQL)
- Big data analytics and processing (Apache Spark, Hadoop)
- Real-time data streaming and processing
- In-memory caches (Redis, Memcached)

Families:

- **R Series (e.g., R8g, R7g, R6g, R5):** Best for high-performance databases (like in-memory databases such as SAP HANA), big data analytics, large in-memory caches (e.g., Redis, Memcached), and enterprise applications requiring substantial memory.
- **X Series (e.g., X2gd, X1e):** Provide extremely high memory capacity and are designed for very large-scale, memory-intensive enterprise workloads.
- **Z1d Series:** Offer high compute capacity combined with high memory, suitable for Electronic Design Automation (EDA) and relational databases.

4. Storage Optimized Instances

- Storage Optimized Instances are designed to deliver high, fast, and low-latency local storage. They are perfect for workloads that require heavy read/write access to large amounts of data stored on the instance.
- **Key Features:**
 - High-speed, low-latency local storage (often NVMe SSDs)
 - Optimized for large sequential I/O operations
 - Enhanced networking capabilities for fast data transfer
- **Use Cases:**
 - Data warehousing and big data analytics
 - High-frequency online transaction processing (OLTP)
 - Distributed file systems
 - Log or data processing applications

- **Families:**
- **I Series (e.g., I4i, I3en, I3):** Optimized for low-latency, high-IOPS transactional workloads. Ideal for NoSQL databases (Cassandra, MongoDB), relational databases, data warehousing, and real-time analytics.
- **D Series (e.g., D3en, D2):** Offer high-density HDD storage for data-intensive workloads. Suitable for distributed file systems (HDFS), large-scale parallel processing (MapReduce), and log processing.
- **H1 Series:** Provide high disk throughput for large-scale data processing and distributed file systems

5. Accelerated Computing Instances

- Accelerated Computing Instances include specialized hardware like GPUs (Graphics Processing Units) or FPGAs (Field Programmable Gate Arrays) to perform specific tasks faster than standard CPUs. These instances are designed for workloads that require heavy computation, such as graphics rendering, machine learning, and scientific simulations.
- **Key Features:**
 - Equipped with GPUs or FPGAs for faster processing
 - High parallel processing power
 - Optimized for compute-intensive and graphics-heavy tasks
- **Use Cases:**
 - Machine learning training and inference
 - Video rendering and transcoding
 - Scientific modeling and simulations
 - Financial risk analysis and high-performance computing (HPC)

- **Families:**
- **P Series (e.g., P5, P4d, P3):** Equipped with NVIDIA GPUs, primarily for machine learning training, high-performance computing (HPC), and deep learning.
- **G Series (e.g., G6, G5, G4dn):** Also use NVIDIA GPUs, but often for graphics-intensive applications (3D rendering, video encoding, virtual workstations), machine learning inference, and game streaming.
- **Inf/Trn Series (e.g., Inf2, Trn1):** Feature AWS Inferentia or Trainium chips, purpose-built for high-performance machine learning inference and training at scale.
- **F1 Series:** Use FPGAs (Field-Programmable Gate Arrays) for custom hardware acceleration, suitable for genomics, financial modeling, and real-time video processing.

AWS EC2 Instance Pricing:

- **Free Tier:**

- Offers certain EC2 instances for free (e.g., **t2.micro** up to 750 hours per month).

- **Payment Options:**

- Multiple ways to pay for EC2 instances depending on usage and requirements.

AWS Cost Calculator:

- Tool provided by AWS to **estimate the cost** of EC2 instances based on instance type and usage.
- **Learn More:** AWS provides detailed resources and the cost calculator for planning expenses.



AWS Pricing Calculator
Estimate the Cost of AWS
Products and Services



Add Services
Search and Add AWS
Services that you Need



Configure Services
Enter the Details of
Your Usage to See
Service Costs



View Estimate Totals
See Estimated Costs
Per Service, Service
Group and Totals

Using AWS Pricing Calculator:

- **Purpose:** Estimate the cost of AWS services.
- **Steps to Use:**
 - Open the **AWS console** and search for **AWS Pricing Calculator**.
 - **Add the service** you want to calculate the cost for.
 - Configure the service based on your requirements.
- **Output:** Get the **estimated cost** in a graph or document format.

A Practical Framework for Choosing an Instance Type

- **Identify Your Bottleneck:** Determine if performance is limited by **CPU**, **memory**, or **disk speed**.
- **Start with General Purpose:** Begin with a **General Purpose** instance (M or T family) for a balanced setup.
- **Monitor Your Application:** Use **Amazon CloudWatch** to track CPU, memory, and disk metrics.
- **Optimize Based on Data:**
 - High CPU usage (>80–90%) → switch to **Compute Optimized (C)** instance
 - Low RAM → switch to **Memory Optimized (R)** instance
 - Disk I/O bottleneck → switch to **Storage Optimized (I)** instance

Aligning Your Choice with a Pricing Model

- **Aligning Instance Choice with Pricing Model:**
- **Instance Type vs Pricing Model:** Choosing hardware is separate from how you pay.
- **On-Demand:**
 - Pay by the second, no long-term commitment
 - Best for **unpredictable workloads**, testing, and development
- **Savings Plans / Reserved Instances:**
 - Commit for **1 or 3 years** for discounts up to 72%
 - Ideal for **steady, predictable workloads**
- **Spot Instances:**
 - Bid on spare EC2 capacity for discounts up to 90%
 - Suitable for **fault-tolerant, non-urgent workloads** (e.g., batch processing, data analysis)
 - Can be **terminated by AWS** with a 2-minute warning

AWS Elastic IP

- A **Static IPv4 address** used in cloud computing is called an **Elastic IP (EIP)**.
- It helps **replace failed instances** quickly by remapping the IP to another instance.
- Each AWS account can **automatically receive an Elastic IP** address.
- The IP remains yours **until you choose to release it**.
- You can **link the Elastic IP to a DNS record** for your domain.
- The EIP acts like a **regular public IPv4 address**, accessible via the internet.
- If an instance **doesn't have a public IPv4 address**, you can associate an EIP with it.
- **Elastic IPs do not support IPv6** addresses.

1. Elastic Pricing for IP Addresses

- **AWS charges a small hourly fee** for Elastic IP (EIP) addresses to ensure efficient use.
- You may be charged if:
 - The **same instance has multiple IP addresses**.
 - The **instance using the IP is stopped or terminated**.
 - The **network interface is not attached** to the IP address.
 - The **IP address is remapped more than 100 times** in a month.
- **Using an Elastic IP is free** while it's **attached to a running instance**.
- **Unused Elastic IPs** (not attached to any instance) cost **\$0.005 per hour**.
- AWS uses this pricing to **encourage efficient use** of limited IP resources.
- **Best practice:** Request only the IPs you need and **release unused ones** back to AWS.

2. Elastic IP Addresses and the AWS Network's Reach

- The **AWS network scope** follows AWS's **geographical regions**.
- **Public IP addresses** assigned to instances are **unique and globally accessible**.
- Example: An instance in the **US East region** gets an IP from that region's AWS IP range and can be **accessed from anywhere in the world**.
- **Elastic IP addresses are region-scoped**, meaning they belong to a specific AWS region.
- By default, you can have **up to five Elastic IP addresses per region**.
- **Private IP addresses** are also region-scoped.
- Instances within the **same region** (across different Availability Zones) can **communicate using private IPs** without **any data transfer charges**.

Working of AWS Elastic IP

Step 1: Allocate an Elastic IP Address – You can easily assign an Elastic IP to your AWS account.

- ***The IP address can come from:***
 - **Amazon's public IP pool** – AWS provides the IP from its global pool of IPv4 addresses.
 - **Your own private IP pool** – Use IP addresses that you've added to your AWS account.
- **Public IPv4 address pool options:**
 - **IPv4 Addresses available on Amazon** – Get an IP from Amazon's maintained pool.
 - **My Collection of Open IPv4 Addresses** – Use IPs from your own added pool (option disabled if no IP pools exist).
 - **Customer-owned IPv4 Address Pool** – Assign IPs from your on-premises network (used with AWS Outposts; unavailable without it).

- **Step 2: Describe your Elastic IP Address –**
You can view detailed information (metadata) of the Elastic IP through the **View Information Portal**.
Metadata includes the **name, description, port numbers**, and other key details.
- **Step 3: Tag an Elastic IP Address –**
Use **tags** to label and identify different Elastic IP addresses, especially when you have **multiple IPs** in your account.
- **Step 4: Connect or Disconnect an Elastic IP Address –**
You can **attach or detach** an Elastic IP from a **network interface** or an **EC2 instance** as needed.
- **Step 5: Release an Elastic IP Address –**
If you no longer need an Elastic IP, **release it** back to AWS.
The IP must **not be attached** to any **EC2 instance, NAT gateway, or Network Load Balancer** before releasing.

Features of AWS Elastic IP

- **Static IPs:** Elastic IP addresses are static and do not change over time.
- **Instance Association:** They are linked to the **primary network interface** of an instance.
- **Public IPv4 Integration:** When attached, the instance's public IPv4 becomes available to other AWS resources.
- **Reassignment Flexibility:** Public IPv4 addresses cannot be reused, but Elastic IPs can be **detached and reassigned** to another instance easily.
- **Persistent Ownership:** The Elastic IP remains with your account **until you release it**, even after detachment.
- **DNS Update:** When associated, the instance's **public DNS hostname updates** to reflect the Elastic IP.
- **IP Source Options:** Elastic IPs can come from **AWS's public IPv4 pool** or your **own custom IP pool**.

- **Limit Exclusion:** IPs from your own pool **don't count** toward AWS Elastic IP limits.
- **Region-Specific:** Each Elastic IP belongs to a specific **region and network border group** and **cannot be moved** across regions.
- **Multi-Service Support:** Can be used with **EC2, Elastic Load Balancer, CloudFront, and Elastic Beanstalk**.
- **Retain Same IP:** Allows you to **keep the same IP** even if an instance is stopped or terminated.
- **Management Tools:** AWS provides tools to **monitor usage, view reports, and automate allocation/release**.
- **Pricing:** Free while **attached to a running instance**, but incurs a **small hourly fee** when unused.
- **Multiple IPs:** You can assign **multiple IPs** to an instance using **Elastic Network Interfaces (ENI)**.
- **High Availability:** Enables **quick IP switching** between instances for fault tolerance and maintenance.
- **Transfer Feature:** You can **transfer Elastic IPs** between **AWS accounts or regions** without downtime.

Limitation of Elastic IP

- **Limited IPv4 resource:** Public IPv4 addresses are scarce and limited worldwide.
- **Default limit:** AWS allows only **five Elastic IP addresses per region** by default.
- **Intended purpose:** Elastic IPs are mainly designed for **replacing failed instances**, not for general communication.
- **Charges for unused IPs:** AWS **charges hourly** for Elastic IPs **not attached** to a running instance.
- **Soft limit:** The **five-IP limit** can be increased only by **requesting AWS support**.
- **Efficient usage needed:** Since IPv4 addresses are limited, users must **allocate only when necessary**.
- **Region-specific:** Elastic IPs **cannot be moved across regions**; a new one must be allocated if an instance moves.
- **Resource management required:** You must **release unused IPs, avoid over-provisioning, and monitor usage** to control costs and prevent waste.

AWS Elastic Beanstalk

- **Web application deployment** usually requires managing many components such as **EC2 servers, load balancers, auto-scaling, application updates, and health monitoring**.
- These setup tasks can be **complex and time-consuming** before the application is ready to serve users.
- **AWS Elastic Beanstalk** simplifies this process by acting as an **orchestration service** that automates deployment and scaling.
- Developers only need to **upload their application code**, and Elastic Beanstalk automatically handles **resource provisioning, load balancing, scaling, and monitoring**.
- It functions as a **Platform as a Service (PaaS)**, which **hides infrastructure complexity** and lets developers focus on **writing code** instead of managing servers.
- This is different from **Infrastructure as a Service (IaaS)** like **Amazon EC2**, where users must **manually manage servers, operating systems, and configurations**.

How Elastic Beanstalk Works

- **Elastic Beanstalk** is a **fully managed AWS service** for easy application deployment and management.
- It allows developers to focus on code **without managing the underlying infrastructure**.
- The basic working process includes:
 - **Create an application** in the Elastic Beanstalk console.
 - **Select and configure an environment** (such as web server or worker environment).
 - **Deploy the application** — Elastic Beanstalk automatically handles **resource provisioning, load balancing, scaling, and monitoring**.

AWS Elastic Beanstalk

Deploy and scale web applications and services



Use AWS Elastic Beanstalk Console

Create an AWS account and answer questions that become inputs to create your environment



Application Code Elastic Beanstalk Platforms

Set up your application (with your code) and runtime (with Elastic Beanstalk platform)

AWS
CloudFormation
Amazon EC2
Instance
S3 Bucket
Amazon
Cloudwatch



Web Application or services

Set up your application (with your code) and runtime (with Elastic Beanstalk platform)

- **Elastic Beanstalk** organizes your deployment using a few key components:

1.Application:

- Acts as a **container for your entire project**.
- Includes all **environments and application versions** related to the project.

2.Application Version:

- Represents a **specific version of your deployable code** (e.g., a .ZIP or .WAR file).
- Stored in an **Amazon S3 bucket** and used for deployment.

3.Environment:

- The **core component** where your application actually runs.
- It consists of **AWS resources** (like EC2 instances, load balancers, etc.) that Elastic Beanstalk
- **provisions and configures** automatically.

When you **launch an environment**, Elastic Beanstalk automatically creates and manages several AWS resources:

- **Amazon EC2 Instances** – Virtual servers that **run your application code**.
- **Auto Scaling Group** – **Adjusts the number of EC2 instances** automatically based on traffic load.
- **Elastic Load Balancer (ELB)** – **Distributes incoming traffic** across EC2 instances for **high availability**.
- **Security Group** – Acts as a **virtual firewall** to control **inbound and outbound traffic** to your instances.
- **Amazon S3 Bucket** – **Stores application files, logs, and other assets** used by Elastic Beanstalk.
- **Amazon CloudWatch Alarms** – **Monitors key performance metrics** and helps track the health of your environment.

Key Features and Benefits

- **Managed Infrastructure** – Automatically **provisions and manages AWS resources**, reducing infrastructure complexity.
- **Auto Scaling** – **Scales up or down** automatically based on workload or traffic, optimizing performance and cost.
- **Health Monitoring** – Offers a **color-coded health dashboard** (Green, Yellow, Red) to easily track application status.
- **Deployment Strategies** – Supports multiple update methods such as **All at once**, **Rolling**, **Rolling with additional batch**, and **Immutable** for **safe, low-downtime deployments**.
- **Developer Productivity** – Enables developers to **focus on coding** and deploy directly using tools like **Git or IDEs**, speeding up development.
- **No Additional Cost** – **Free to use** Elastic Beanstalk itself; you only pay for the **AWS resources** (e.g., EC2, ELB, S3) it uses.

Elastic Beanstalk-Supported Platforms

- AWS Elastic Beanstalk supports multiple **programming platforms** for deploying applications.
- It **automatically manages** the architecture, servers, and other resources needed to run the code.

Supported programming platforms include:

- Java
- Node.js
- Python
- PHP
- Ruby
- Go

Platforms for Application Servers Provided by Elastic Beanstalk are

- [Tomcat](#)
- [Docker](#)
- Elastic Beanstalk ensures that each platform is **kept up-to-date** and **optimized for AWS environments**.

How to Deploy an Application Using Elastic Beanstalk

Step 1: Open Elastic Beanstalk in AWS Console

- Go to the **AWS Management Console**.
- In the **search bar**, type “**Elastic Beanstalk**.”
- Select **Elastic Beanstalk** from the drop-down results.
- You’ll be redirected to the **Elastic Beanstalk dashboard**.
- Click on “**Create Application**” to start setting up your application.
- You can **launch a sample application** or upload your own code.

aws

Services

Q Elastic Beanstalk

X

New EC2 Experience

Tell us what you think

EC2 Dashboard

EC2 Global View

Events

Tags

Limits

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Images

AMIs

AMI Catalog

Elastic Block Store

Unlabeled

Search results for 'Elastic Beanstalk'

Services

See all 12 results

Elastic Beanstalk

Run and Manage Web Apps

Top features

Applications

Environments

Elastic Transcoder

Easy-to-Use Scalable Media Transcoding

Elastic Container Service

Highly secure, reliable, and scalable way to run containers

Elastic Container Registry

Fully-managed Docker container registry : Share and deploy container software, publ...

Features

See all 26 results

Applications

Elastic Beanstalk feature

Actions

Launch instances

< 1 >

⊙

Alarm status

Availability Zone

Public IP

No alarms + ap-south-1b -

No alarms + ap-south-1b -

⊙ X

aws

Services

Search

[Alt+S]

Mumbai

naveen

Elastic Beanstalk

×

Environments

Applications

Change history

Amazon Elastic Beanstalk

End-to-end web application management.

Amazon Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker on familiar servers such as Apache, Nginx, Passenger, and IIS.

Get started

Easily deploy your web application in minutes.

Create Application

How it works

You simply upload your code and Elastic Beanstalk automatically handles the deployment, from capacity provisioning, load balancing, and automatic scaling to web application health monitoring, with ongoing fully managed patch and security updates. [Learn more](#)

Pricing

There's no additional charge for Elastic Beanstalk. You pay for Amazon Web Services resources that we create to store and run your web application, like Amazon S3 buckets and Amazon EC2 instances.

CloudShell

Feedback

Language

© 2023, Amazon Web Services India Private Limited or its affiliates.

Privacy

Terms

Cookie preferences

- **Step 3: Configure Application Information**
- Provide the **Application Name** and optional **key-value pairs (tags)** for identification.
- Choose the **platform** based on the **programming language** used (e.g., Java, Python, Node.js, etc.).
- Select the **platform branch** and **platform version** suitable for your application.
- Under **Application Code**, choose one of the following options:
 - **Use a Sample Application:** Deploy a pre-built sample app provided by AWS (e.g., Java sample app).
 - **Upload Your Own Code:** Upload your application as a **.ZIP or .WAR** file from your **local computer** or **S3 bucket**.
- Ensure the **version label** you provide is **unique** for each deployment.

aws

Services

Search

[Alt+S]

Mumbai

naveen

Elastic Beanstalk

Environments

Applications

Change history

Elastic Beanstalk > Getting started

Create a web app

Create a new application and environment with a sample application or your own code. By creating an environment, you allow Amazon Elastic Beanstalk to manage Amazon Web Services resources and permissions on your behalf. [Learn more](#)

Application information

Application name

Gfg-application

Up to 100 Unicode characters, not including forward slash (/).

Application tags

Apply up to 50 tags. You can use tags to group and filter your resources. A tag is a key-value pair. The key must be unique within the resource and is case-sensitive. [Learn more](#)

Key

Value

Remove tag

Add tag

50 remaining

CloudShell

Feedback

Language

© 2023, Amazon Web Services India Private Limited or its affiliates.

Privacy

Terms

Cookie preferences

aws

Services

Search

[Alt+S]

Mumbai

naveen

Elastic Beanstalk

×

Environments

Applications

Change history

Platform

Platform

Java

▼

Platform branch

Corretto 17 running on 64bit Amazon Linux 2

▼

Platform version

3.4.6 (Recommended)

▼

Application code

☒ Sample application

Get started right away with sample code.

☐ Upload your code

Upload a source bundle from your computer or copy one from Amazon S3.

Cancel

Configure more options

Create application

CloudShell

Feedback

Language

© 2023, Amazon Web Services India Private Limited or its affiliates.

Privacy

Terms

Cookie preferences

- **Step 4:** In this step, We do not need to do anything AWS Elastic Beanstalk will take care of the resource required to deploy the web application like
 - [EC2 instance](#)
 - [Security Group](#)
 - [S3 bucket](#)
 - [Environment](#)
 - [Load Balancer Listener](#)
 - [CloudWatch Alarms](#)
 - [Auto Scaling Group](#)
 - [Elastic load balancer \(ELB\)](#)

8:40pm Successfully launched environment: GettingStarted-env

8:39pm Environment health has transitioned from Pending to Ok. Initialization completed 16 seconds ago and took 5 minutes.

8:36pm Added instance [i-045eb69a24818d1d4] to your environment.

8:36pm Waiting for EC2 instances to launch. This may take a few minutes.

8:35pm Created EIP: 34.230.236.246

8:34pm Created security group named:
eb-dv-e-sbj4gzf2dm-stack-AWSEBSecurityGroup-KATGTRO6V1J9

8:34pm Environment health has transitioned to Pending. Initialization in progress (running for 8 seconds). There are no instances.

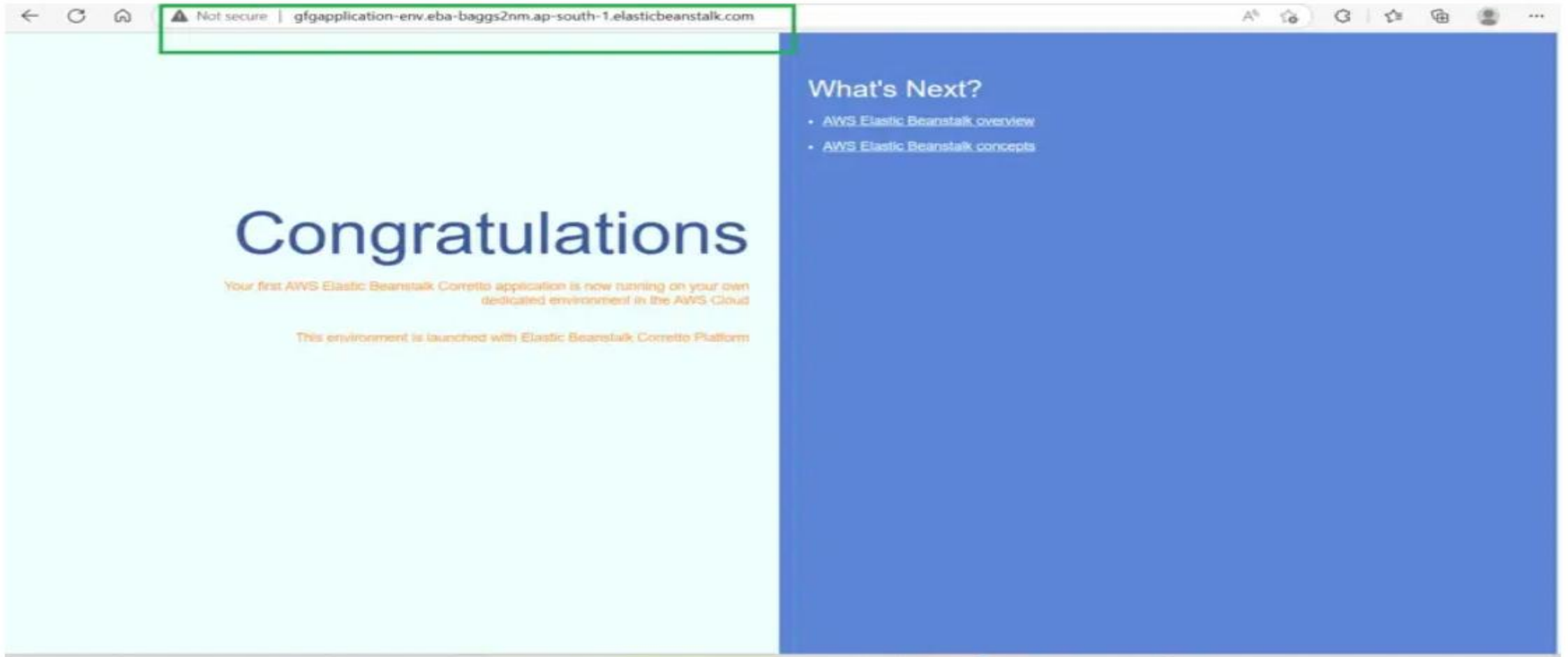
8:34pm Using elasticbeanstalk-us-east-1-270205402845 as Amazon S3 storage bucket for environment data.

8:34pm createEnvironment is starting.

- **Step 5: Verify the Deployment**

- After deployment, you will see your **environment name** (e.g., *Gfgapplication-env*) on the Elastic Beanstalk dashboard.
- The **application URL** is displayed — this link allows you to **access your app on the internet**.
- The **Health Status** shows a **green “OK” indicator**, meaning the application is **running smoothly without issues**.
- In the **Recent Events** section, you can view the **resources and architecture** (like EC2 instances, load balancer, and S3 bucket) that **Elastic Beanstalk automatically created** to deploy your application successfully.

- **Step 6:** The very last and most important step requires us to use the URL provided by the Elastic Beanstalk to visit our Java web application. as seen in the image below.



Ways to Interact with Elastic Beanstalk

1.AWS Management Console –

- A **graphical web interface** used to create, manage, and monitor applications visually.
- Ideal for **beginners** and for **visual monitoring** of application health and resources.

2.EB CLI (Elastic Beanstalk Command Line Interface) –

- A **command-line tool** for developers to manage environments and deploy apps directly from the terminal.
- Common commands:
 - eb init – Initializes a project directory.
 - eb create – Creates a new environment.
 - eb deploy – Deploys a new version of the application.

3.AWS SDKs (Software Development Kits) –

- Enable developers to **interact programmatically** with Elastic Beanstalk.
- Useful for **automating deployments**, managing environments, and integrating Elastic Beanstalk with your own code or applications.

AWS Fargate

Introduction

- **AWS Fargate** is a **serverless compute engine** for running containers.
- It **removes the need to manage servers** manually.
- Fargate is a part of **Amazon Web Services (AWS)**.
- It is designed to **simplify containerized application deployment**.
- AWS handles **provisioning, management, and automatic scaling** of infrastructure.
- Developers can **focus on application code** instead of managing servers.

What is AWS Fargate?

- **AWS Fargate** is a **fully managed service** from AWS.
- It allows you to **run containerized applications** without managing servers or clusters.
- Fargate works with both **AWS Elastic Container Service (ECS)** and **Elastic Kubernetes Service (EKS)**.
- It **abstracts infrastructure management**, letting developers focus on application development and deployment.
- You **pay only for the CPU and memory resources** that your containers actually use.
- AWS automatically **provisions the required infrastructure** based on workload.
- Fargate is a **cost-effective and efficient solution** for simplifying container management.

What is Serverless and Why is it Game-Changing?

- **Serverless computing** is a cloud model that lets developers run code without managing servers.
- It **abstracts the underlying infrastructure**, allowing developers to focus on writing and deploying applications.
- **AWS Fargate** brings the **serverless model to containerized applications**.
- In traditional container services, you need to manage **EC2 instances and clusters** manually.
- Fargate **automatically allocates compute resources** based on defined requirements (CPU, memory).
- It **handles infrastructure management** entirely on behalf of the user.
- The **game-changing benefit** is that developers can **build and scale applications** without worrying about servers.
- This approach **reduces operational overhead and lowers costs** for businesses.

AWS Fargate Features

- **Serverless Container Management:**
 - No need to manage or provision servers manually.
 - AWS Fargate automatically handles infrastructure provisioning and scaling.
- **Pay-as-You-Go Pricing:**
 - You pay only for the **CPU and memory** your containers actually use.
 - Enables **cost-efficient** operations.
- **Seamless Integration with ECS and EKS:**
 - Works smoothly with **Amazon Elastic Container Service (ECS)** and **Elastic Kubernetes Service (EKS)**.
 - Supports familiar AWS container orchestration tools.
- **Automatic Scaling:**
 - Automatically **scales resources** based on application demand.
 - Adjusts compute capacity **up or down** depending on traffic.
- **Security and Compliance:**
 - Integrates with **AWS Identity and Access Management (IAM)** and **Virtual Private Cloud (VPC)**.
 - Ensures **secure application deployment** and **data protection**.

Components of AWS Fargate

- **Clusters:**

- Logical groups that organize tasks and services in Fargate.
- Act as the environment where applications run.

- **Task Definitions:**

- Configuration files that define containers and their settings.
- Serve as **blueprints** for Fargate applications, specifying parameters like CPU, memory, and image details.

- **Tasks:**

- The **running instances** of task definitions.
- Multiple tasks can be created from a single task definition within a cluster.
- Tasks can run **individually or in parallel**, depending on requirements.

- **Services:**

- Manage and maintain running tasks in Fargate.
- Ensure tasks are **automatically replaced** if they fail or stop.
- Support **scaling** and **continuous availability** of applications.

How AWS Fargate Works?

- AWS Fargate is a **serverless compute engine** for running containers.
- You don't need to **manage or maintain servers**.
- Just **set your application's requirements** (CPU, memory, storage).
- Fargate **automatically provisions and manages** the infrastructure.
- It lets you **focus on your application**, not on server management.
- **Package your application** and its dependencies into a **Docker image**.
- **Configure** the required settings such as CPU, memory, and networking.
- Fargate **deploys the application automatically** on virtual machines.
- You **pay only for the resources** (CPU and memory) your application uses.



Build your
container image



Define and deploy
the EC2 Instances



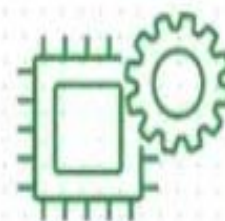
Provision and manage
compute and memory
resources



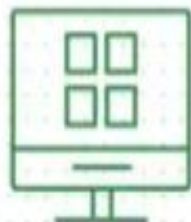
AWS Fargate



Build container image



Define memory and
compute resources
required



Isolate applications in
separate VMs



Run and manage
both applications
and infrastructure



Pay for EC2
Instances



Run and manage
applications



Pay for requested compute
resources when used.
Application isolation
by design

AWS Fargate Use Cases

- **Large Workloads with Minimal Effort:**
 - Ideal for handling large workloads automatically.
 - Manages infrastructure, scaling, and monitoring without manual work.
- **Microservice Applications:**
 - Suitable for deploying **microservices, APIs, web apps, and AI/ML models.**
 - Enables **easy scaling and fast deployment** of containerized applications.
- **Small Workloads with Occasional Spikes:**
 - Efficiently **scales resources up or down** based on demand.
 - Helps **reduce costs** for applications with variable or low traffic.
- **Batch Jobs:**
 - Great for **batch processing tasks.**
 - Automatically starts and stops containers as needed, removing manual management.

Benefits of AWS Fargate

- **Cost Efficiency:**
 - Uses a **pay-as-you-go** model — you pay only for the CPU and memory your containers use.
 - Reduces costs by removing the need for idle servers.
- **No Infrastructure Management:**
 - No need to **manage EC2 instances or clusters**.
 - AWS handles provisioning, scaling, and patching automatically.
- **Seamless Scaling:**
 - **Automatically scales** your applications based on demand.
 - Handles traffic spikes or drops **without manual effort**.
- **Simplified Operations:**
 - Works seamlessly with **ECS and EKS** for easy deployment and management.
 - Lets developers **focus on application code and performance**, not servers.
- **Enhanced Security:**
 - Integrated with **AWS IAM** for access control.
 - Uses **Amazon VPC** for a secure and isolated network environment.

AWS Fargate Drawbacks

- **Cost for Long-Running Tasks:**
 - Fargate can be **more expensive** than EC2 for workloads that run continuously.
 - EC2 instances may be **more cost-effective** for always-on applications.
- **Less Control Over Infrastructure:**
 - Since Fargate is **fully managed**, users have **limited control** over server configurations and performance tuning.
 - Not ideal for teams needing deep infrastructure customization.
- **Limited Availability in Some Regions:**
 - Fargate is **not available** in every AWS region.
 - Users must **check regional availability** before deployment.

AWS Fargate Pricing

- **Pricing Based on Resource Usage:**
 - Charges depend on the **CPU, memory, storage**, and the **AWS region** used.
 - You pay only for the **resources consumed per second** while your containers are running.
- **Example (Asia Pacific – Mumbai Region):**
 - **vCPU per hour:** \$0.04256
 - **Memory per GB per hour:** \$0.004655
- **AWS Fargate Spot:**
 - Offers up to **70% lower prices** by using **unused AWS capacity**.
 - Ideal for **cost-conscious** users and **flexible workloads**.

AWS Fargate vs AWS EC2 Instance

AWS Fargate	AWS EC2 Instance
AWS Fargate is an serverless compute engine for the an applications which need to be run in the containerization form without worrying about the underlying infrastructure.	<u>AWS EC2</u> instance is most preferable for the virtualization application but manual provisions of the underlying infrastructure is required.
Containers will be isolated from each other where no data breach will be happen.	You have complete control so the isolation is also depends in you hands more careful configuration is required.
AWS Fargate scales automatically the applications by depending on the incoming traffic.	AWS EC2 will not be scaled automatically it should be integrated with other AWS service for autoscaling of the instances.
AWS Fargate is for short-running tasks.	EC2 consists of long-running containers.

AWS Fargate vs AWS Lambda

Feature	AWS Fargate	AWS Lambda
Use Case	Ideal for running containerized applications and managing stateful workloads.	Ideal for running short, stateless functions triggered by events.
Type	Container-as-a-Service (CaaS)	Function-as-a-Service (FaaS)
How They Run	Runs containers on EC2 instances managed by AWS.	Runs individual code functions without server management.
Event Triggers	No built-in event trigger. Runs continuously based on container configuration.	Event-driven; responds to triggers like S3 object uploads.
Best For	Stateful applications, microservices, batch jobs, large workloads.	Short, event-driven applications (e.g., API backend, real-time processing).
Pricing Model	Pay-as-you-go model based on resources consumed (CPU, memory).	Pay-as-you-go model based on execution time (time and memory used).
Scaling	Automatically scales containers based on demand.	Automatically scales based on the frequency of the events.
Resource Management	Manages resources like CPU and memory based on container configuration.	Managed automatically for short-term code execution.

Amazon Elastic Container Service (ECS)

Introduction

- **Amazon ECS** is a **fully managed container orchestration service** by AWS.
- It allows users to **run Docker-based applications** in containers on a cluster of EC2 instances.
- ECS simplifies **deployment, management, and scaling** of containerized applications.
- It supports multiple **launch types**, including **AWS Fargate**.
- The **Fargate launch type** removes the need to manage EC2 instances manually.
- Developers can **focus on application development**, while AWS handles **cluster management and resource provisioning** automatically.

The Core Components

1.Task Definition:

- Acts as the **blueprint** for your application.
- Written in **JSON format**, describing how to run a Docker container.
- Specifies:
 - Docker image (e.g., nginx:latest or an image from Amazon ECR)
 - **CPU and memory** allocation
 - **Port mappings**
 - **Environment variables, logging, and IAM roles** for permissions

2.Task:

- A **running instance** of a Task Definition.
- If the Task Definition is the blueprint, the Task is the **actual container** running the application.

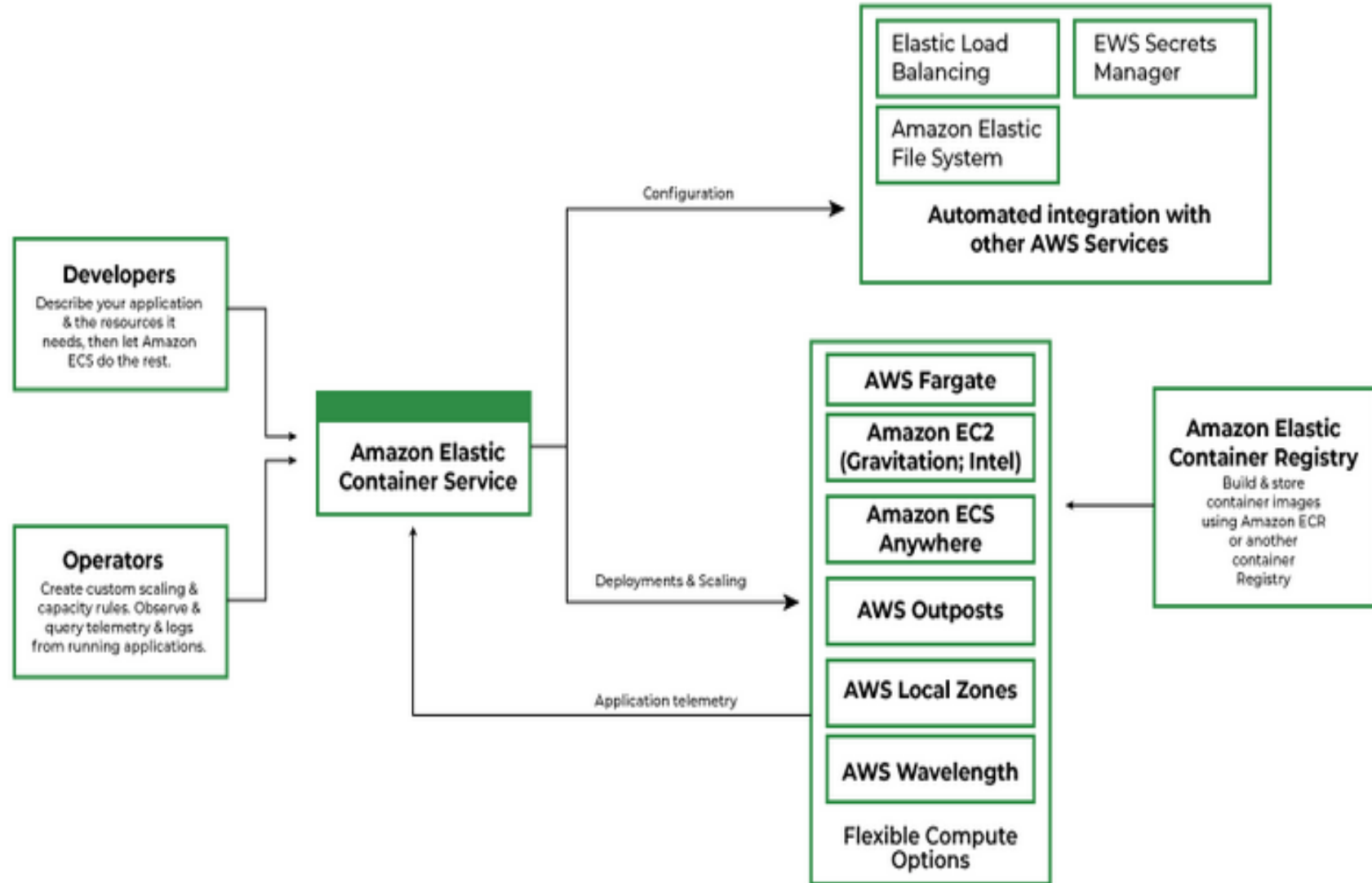
3.Service:

- Used for **long-running applications**.
- Ensures the **desired number of tasks** are always running.
- **Automatically replaces failed tasks** (self-healing).
- Supports **Load Balancing** via Application Load Balancer (ALB).
- Enables **Auto Scaling** based on metrics like CPU or traffic load.

4.Cluster:

- A **logical group of resources** where ECS runs tasks and services.
- Provides the **CPU, memory, and networking** infrastructure for your containers.

How Elastic Container Service Works?



- **Amazon ECS** is a **fully managed service by AWS** used to **deploy and manage Docker-based containers**.
- It allows applications to **scale up or down automatically** based on traffic demand.
- Containers in ECS run inside **Amazon EC2 instances**.
- **Key Components:**
- **Container:**
 - A **package** that includes an application and all its **dependencies, libraries, and configurations**.
 - Containers are **portable, flexible, and scalable**.
 - They ensure the application **runs consistently** across different systems and environments.
- **Docker:**
 - A **software platform** that helps in creating, deploying, and running applications inside containers.
 - It **automates container installation and deployment**, mainly on **Linux systems**.
- **Cluster:**
 - A **logical group of EC2 instances** that work together to run containerized applications.
- **Container Instance:**
 - Each **EC2 instance** within an ECS Cluster is called a **Container Instance**.
 - It serves as the **host machine** for running containers.

The Most Important Choice: Fargate vs. EC2 Launch Types

- **1. AWS Fargate (Serverless Launch Type)**
- **What it is:**
 - A **serverless compute engine** for containers.
 - You define your app in a **Task Definition**, and Fargate handles container launching and management.
 - No need to manage or provision servers.
- **How it works:**
 - You **pay per second** for the **CPU and memory** your containers use.
- **Best for:**
 - **Modern applications** like microservices, web apps, and batch jobs.
 - Developers who want to **focus on code**, not infrastructure.
 - Provides the **fastest and simplest** way to run containers on AWS.

2. EC2 Launch Type

- **What it is:**

- You **manage the EC2 instances** that run your containers.
- The ECS **agent** on each EC2 instance registers it to the ECS Cluster.

- **How it works:**

- The **ECS scheduler** places tasks on EC2 instances in your cluster.
- You have **full control** over the instance type, OS, and configurations.

- **Best for:**

- Applications requiring **specific compliance or customization**.
- Workloads needing **GPUs or specialized hardware**.
- Users wanting **fine-grained control** over the hosting environment.

Amazon ECS Terminologies and Components

- **Amazon ECS Overview**
- **Amazon ECS is a fully managed service for deploying, managing, and scaling containers easily.**
- **1. Provisioning**
- **Tools used to set up and manage ECS resources:**
 - **AWS CLI:** Command-line tool for direct ECS control.
 - **AWS Copilot:** Simplifies ECS workflows and deployments.
 - **AWS Management Console:** Graphical web interface for managing ECS.
 - **AWS CDK & SDK:** Allows infrastructure as code and custom integrations.

2. Controller

- **ECS Scheduler** acts as the controller.
- It **manages and distributes tasks** across clusters.
- Ensures **balanced workloads** and **high availability**.

3. Capacity Options

- ECS supports multiple hosting options for containers:
 - **Amazon EC2 Instances:** Run containers on customizable virtual machines.
 - **AWS Fargate:** Serverless option with no infrastructure to manage.
 - **On-Premises Compute:** Extend ECS to **on-premises servers** for **hybrid cloud** setups.

Amazon ECS terminology and components

Provisioning

Amazon Web Services
Command Line Interface

Copilot

Management
console

Amazon Web Services
Cloud Developer Kit

Amazon Web Services
Software Developer Kit



Amazon ECS scheduler



Controller



Amazon EC2 instances

Amazon Web Services Fargate

On-premises compute

Capacity options



How ECS Auto Scaling Works

- ECS offers **two types of auto scaling** to manage performance and cost efficiently:
- **1. Task-Level Scaling (Service Auto Scaling):**
- Automatically **adds or removes ECS tasks** based on application demand.
- Example rule: *If average CPU usage exceeds 75%, add two more tasks.*
- Ideal for **handling traffic spikes** and ensuring smooth performance.
- Helps maintain the **desired service availability** without manual intervention.

- **2. Infrastructure-Level Scaling (Cluster Auto Scaling):**
- Applies **only to the EC2 launch type**.
- Automatically **adds or removes EC2 instances** in the cluster depending on task requirements.
- Ensures that you **use and pay only for the EC2 instances** actually needed.
- Helps maintain **cost efficiency and resource optimization**.

Common Use Cases for ECS

Microservices Architecture:

- Ideal for breaking a **monolithic application** into smaller, **independently deployable microservices**.
- Each microservice can have its **own ECS Service**, scaling rules, and deployment pipeline.

Web Applications:

- Run **scalable and highly available web apps** using ECS.
- Use an **Application Load Balancer (ALB)** in front of ECS Services to distribute traffic evenly.

Batch Processing:

- Suitable for **short-lived, data-heavy tasks** like financial modeling, scientific simulations, or ETL jobs.
- These can be **scheduled as individual ECS Tasks**.

CI/CD Pipelines:

- Run **build, test, and deployment jobs** inside containers.
- Enables a **consistent, automated, and repeatable** development pipeline.

Advantages of ECS

- **Advantages of Amazon ECS:**
- **Scalability:** Automatically scales applications based on demand.
- **High Availability:** Ensures your applications stay running with fault tolerance.
- **Cost-Effective:** You pay only for the resources you use.
- **Integration:** Works well with AWS services like **ECR, Fargate, CloudWatch, and IAM.**
- **Security:** Offers strong security features like **IAM roles, VPC isolation, and data encryption.**
- **Limitations:**
- Requires some **expertise** to set up and configure.
- May not suit applications needing **high customization** or **complex networking.**

Ways to Provision and Manage ECS

- **AWS Management Console:**
 - A **web-based graphical interface** for managing ECS.
 - Best for **beginners** and for **visual monitoring**.
- **AWS Copilot CLI:**
 - A **developer-friendly command-line tool**.
 - Simplifies **building, releasing, and operating** containerized apps on ECS.
- **AWS CDK (Cloud Development Kit):**
 - Lets you define ECS infrastructure **as code** using languages like **Python, TypeScript, or Go**.
 - Ideal for **automation and repeatable deployments**.
- **AWS CLI:**
 - The **standard command-line interface** for ECS.
 - Provides **direct and detailed control** over ECS resources.

AWS Command Line Interface(AWS CLI)

Cluster Commands

- **create-cluster** – Create a new ECS cluster.
- **delete-cluster** – Delete an existing ECS cluster.
- **describe-clusters** – View details of one or more clusters.
- **list-clusters** – List all ECS clusters.
- **update-cluster-settings** – Modify cluster settings.

Task and Task Definition Commands

- **register-task-definition** – Register a new task definition.
- **deregister-task-definition** – Remove a task definition.
- **describe-task-definition** – Show details of a task definition.
- **list-task-definitions** – List all registered task definitions.
- **list-task-definition-families** – List task definition families.
- **run-task / start-task** – Start tasks from a task definition.
- **stop-task** – Stop a running task.
- **describe-tasks** – Describe one or more tasks.
- **list-tasks** – List running tasks in a cluster.

Capacity Provider Commands

- **create-capacity-provider** – Create a new capacity provider.
- **describe-capacity-providers** – View capacity provider details.
- **put-cluster-capacity-providers** – Attach capacity providers to a cluster.
- **vContainer Instance Commands**
- **register-container-instance** – Add a container instance to a cluster.
- **deregister-container-instance** – Remove a container instance.
- **describe-container-instances** – View container instance details.
- **list-container-instances** – List container instances in a cluster.
- **update-container-agent** – Update the ECS container agent.
- **update-container-instances-state** – Change instance status.

Account and Attribute Commands

- **put-account-setting / delete-account-setting** – Manage account settings.
- **put-account-setting-default** – Apply default account settings.
- **list-account-settings** – List account settings.
- **put-attributes / delete-attributes** – Manage ECS resource attributes.
- **list-attributes** – List ECS resource attributes.

Tagging and Metadata

- **tag-resource / untag-resource** – Add or remove tags from resources.
- **list-tags-for-resource** – View tags for a specific resource.

Task Set Commands

- **create-task-set / delete-task-set** – Create or delete a task set.
- **describe-task-sets / update-task-set** – Manage task sets.

State Change and Monitoring

- **submit-task-state-change** – Report a task state change.
- **submit-container-state-change** – Report container state change.
- **submit-attachment-state-changes** – Report attachment changes.
- **discover-poll-endpoint** – Get ECS agent update endpoints.
- **wait** – Wait until a specific ECS condition is met.

PART B

AWS Simple Storage Service (AWS S3)

- **AWS provides multiple storage options** to suit different project requirements and data types.
- These services are designed to handle **confidential data, frequently accessed data, and long-term archived data**.
- You can choose the right storage type based on **access frequency, performance, and cost needs**.
- Major types of AWS storage services include:
 - **Object Storage (Amazon S3)**: Stores unstructured data like images, videos, and backups.
 - **File Storage (Amazon EFS)**: Provides scalable file storage for Linux-based workloads.
 - **Block Storage (Amazon EBS)**: Offers high-performance storage for EC2 instances.
 - **Backup Services**: Securely back up data across AWS environments.
 - **Data Migration Services**: Move data efficiently to and from AWS storage systems.

- **What is Amazon S3?**
- **Amazon S3 (Simple Storage Service)** is a cloud-based storage service provided by AWS.
- It stores different types of files such as **photos, videos, audio, and documents** as **objects**.
- Users can **store and retrieve unlimited data** anytime and from anywhere on the internet.
- It offers **high scalability, durability, and security** for data storage.
- S3 easily **integrates with other AWS services** for backup, analytics, and application use.

- **What is Amazon S3 Used For?**

- Amazon S3 is widely used for storing and managing data in the cloud with high scalability and security.
- **Data Storage:**
Used to store and retrieve large or small amounts of data for different applications.
- **Backup and Recovery:**
Helps organizations back up critical data and recover it easily when needed.
- **Hosting Static Websites:**
Can host static websites using HTML, CSS, and other web files with low cost and fast access.
- **Data Archiving:**
With **Amazon S3 Glacier**, it stores long-term or rarely accessed data at a low cost.
- **Big Data Analytics:**
Acts as a **data lake** for storing structured and unstructured data and integrates with AWS Analytics and Machine Learning services.

- **What is an Amazon S3 Bucket?**
- An **Amazon S3 bucket** is a **storage container** used to store data (called objects) in AWS S3.
- It can store **files like text, images, audio, and videos** securely in the AWS Cloud.
- Each bucket has a **globally unique name** across all AWS users.
- Buckets can be configured with **Access Control Lists (ACLs)** to manage permissions and data access.

- **How Does Amazon S3 Work?**

- Amazon S3 organizes data into **unique S3 buckets**.
- Users can **store objects (files, data, media)** inside these buckets.
- Each bucket can be **customized with access controls** to manage permissions.
- S3 supports **versioning**, allowing you to keep multiple versions of the same object.
- **Lifecycle management** helps automatically move or delete data based on defined rules.
- S3 automatically **scales** to handle large amounts of data efficiently.

- **1. Amazon S3 Buckets and Objects**
- **Amazon S3 Buckets**
- Data in S3 is stored inside **containers called buckets**.
- Each bucket has its own **policies and configurations** for better control.
- **Bucket names must be unique** across all AWS accounts.
- A bucket can be seen as a **parent folder** containing objects.
- Each AWS account can create **up to 100 buckets** (limit can be increased upon request).

- **Amazon S3 Objects**
- **Objects** are the **fundamental data entities** stored inside S3 buckets.
- You can store **unlimited objects** in a bucket.
- Each object can be **up to 5 TB** in size.
- An object includes:
 - **Key:** The unique name of the object.
 - **Version ID:** Identifies object versions (if versioning is enabled).
 - **Value:** The actual data or file content.
 - **Metadata:** Information about the object (e.g., file type).
 - **Subresources:** Additional features like access control.
 - **Access Control Information:** Defines who can access the object.
 - **Tags:** Labels used for organization and cost tracking.

- **2. Amazon S3 Versioning and Access Control**

- **S3 Versioning**

- **Versioning** keeps track of all previous versions of objects in an S3 bucket.
- **Not enabled by default** — must be manually turned on.
- Once enabled, it applies to **all objects** in that bucket.
- Helps prevent **accidental overwrites or deletions** of files.
- Each version of a file has a **unique Version ID**.
- **Increases storage costs** since all versions are stored (e.g., 10 copies of a 1 GB file = 10 GB total storage).
- Useful for **data recovery** and maintaining a change history.

- **Access Control Lists (ACLs)**
- **ACLs** define who can access your S3 buckets and objects.
- Each bucket has its own ACL document specifying **permissions**.
- Used to **grant access** to users or accounts outside your AWS account.
- You can manage ownership and access more easily using **S3 Object Ownership** (a bucket-level feature).
- ACLs can be **enabled or disabled** based on your security preferences.

- **3. Bucket Policies and Lifecycle Rules**

- **Bucket Policies**

- **Bucket policies** control access to S3 buckets **within your AWS account**.
- They define **which users or services** can access the bucket and **what actions** they can perform.
- Each bucket has its **own policy document** written in JSON format.
- Used to manage **permissions** like read, write, or delete access for specific users or roles.
- Helps maintain **security and compliance** across S3 resources.

- **Lifecycle Rules**
- **Lifecycle rules** help manage data efficiently and reduce storage costs.
- Automatically **move old files** to cheaper storage classes (like **S3 Glacier**) or **delete** them after a set period.
- Ideal for managing **infrequently accessed** or **archival data**.
- Supports **automated data transitions** between S3 storage tiers based on age or usage.
- Ensures **cost optimization** and efficient data retention management.

4. Keys and Null Objects

Keys

- A **key** is a **unique identifier** for each object stored in an S3 bucket.
- It acts like a **file path** within the bucket.
- Example: In bucket '**ABC**', if your file is stored at `javaPrograms/GFG.java`, then the **object key** is `javaPrograms/GFG.java`.
- Keys help **organize and locate** objects easily within the bucket.

Null Objects

- A **null object** appears when **versioning is suspended** in an S3 bucket.
- These objects have a **null Version ID**.
- They represent files uploaded **after versioning was turned off**.
- Useful for identifying **unversioned objects** in buckets that previously had versioning enabled.

- **How to Use an Amazon S3 Bucket**
- You can use Amazon S3 buckets by following these simple steps:
- **Step 1: Create an S3 Bucket**
- Log in to your **AWS Management Console**.
- Search for “**S3**” and click to open the service.
- Click “**Create bucket**” and configure the required settings such as:
 - Bucket name (must be globally unique)
 - Region
 - Permissions and access options

Step 2: Upload Objects

- After creating the bucket, you can **upload objects** (files, images, videos, etc.) either through:
 - **AWS Management Console**, or
 - **AWS CLI** using the command:

```
pgsql
```

```
aws s3 cp <local-file-path> s3://<bucket-name>/
```

- **Step 3: Set Permissions**

- Control who can access your bucket and its objects.
- You can make your bucket **public** or **private** (by default, it's private).
- Use **bucket policies** or **Access Control Lists (ACLs)** to manage access.

- **Step 4: Manage Lifecycle Rules**

- Enable **Lifecycle Management** to automate data movement.
- Define rules to **transition** data to cheaper storage classes (e.g., Glacier) or **delete** old data after a set time.

- **Step 5: Monitor and Analyze**

- Enable **S3 Access Logging** to track who accesses your bucket.
- You can also integrate with **Amazon CloudWatch** for deeper insights into usage and performance.

- **Types of Amazon S3 Storage Classes**
- Amazon S3 offers multiple **storage classes** designed to balance **cost, performance, and data access frequency**. Each class serves a different use case.
- **1. S3 Standard**
- **Use Case:** Frequently accessed data
- **Features:**
 - High durability (99.999999999%) and availability
 - Ideal for active data such as websites, mobile apps, and big data analytics
- **Cost:** Highest among S3 classes due to frequent access

- **2. S3 Standard–Infrequent Access (S3 Standard-IA)**
- **Use Case:** Infrequently accessed data (e.g., backups, log archives)
- **Features:**
 - Lower cost than Standard
 - Retrieval incurs additional per-GB fees
 - Suitable for data that must remain available but is rarely accessed
- **3. S3 Intelligent-Tiering**
- **Use Case:** Unpredictable or changing data access patterns
- **Features:**
 - Automatically moves data between **frequent** and **infrequent access** tiers
 - Reduces cost without performance impact
 - No retrieval fee

- **4. S3 One Zone–Infrequent Access (One Zone-IA)**
- **Use Case:** Non-critical, infrequently accessed data
- **Features:**
 - Data stored in a **single Availability Zone**
 - Cheaper than Standard-IA
 - Lower durability (no multi-AZ redundancy)
 - Best for re-creatable or easily replaceable data
- **5. Reduced Redundancy Storage (RRS)**
- **Use Case:** Non-essential or temporary data
- **Features:**
 - Durability: **99.99%** (lower than other classes)
 - AWS no longer recommends RRS for most use cases
 - Suitable for non-critical, reproducible content

- **How to Upload and Manage Files on Amazon S3**
- To upload and manage files in **Amazon S3**, you first need to **create an S3 bucket**. Once the bucket is ready, you can store and organize your files efficiently.
- **1. Uploading Files**
- You can upload files to your S3 bucket using any of the following methods:
- **AWS Management Console:** Use the graphical interface to drag and drop files or folders directly.
- **AWS CLI:** Use the command line for uploading files.
Example:
 - `aws s3 cp <local-file-path> s3://<bucket-name>/`
- **AWS SDKs:** Integrate S3 operations into your applications using programming languages like Python, Java, or Node.js.

- **2. Managing Files**

- Once uploaded, you can efficiently manage your files by:
- **Organizing files into folders** inside the S3 bucket.
- **Setting access permissions** to make files public or private using **ACLs** or **Bucket Policies**.
- **Enabling Versioning** to maintain older versions of files and recover deleted or modified data.
- **Applying Lifecycle Policies** to automatically transition data to lower-cost storage classes or delete it after a certain period.

- **How to Access Amazon S3 Bucket**
- You can access and work with **Amazon S3 buckets** in multiple ways depending on your preference and use case:
- **AWS Management Console**
- **AWS CLI (Command Line Interface)**
- **Programming Scripts (using the Boto3 library in Python)**

- **1. AWS Management Console**

- The **AWS Management Console** is a web-based interface that allows you to manage your S3 buckets easily.

Steps:

- Sign in to your **AWS account**.
- Open the **AWS Management Console**.
- Navigate to **Amazon S3 → Buckets**.
- Select or create a bucket to upload, manage, or access your files.
- *(Path: AWS Console → Amazon S3 → S3 Buckets)*

- **2. AWS CLI Commands**

- You can access S3 from your terminal using the **AWS Command Line Interface (CLI)**.

Steps:

1. Install the **AWS CLI** tool.

2 Configure your AWS account using:

- ***aws configure***

3 To list all your S3 buckets, use:

4. You can also upload, download, or delete files using CLI commands.

- **3. Programming Scripts (Using Boto3 Library in Python)**
 - You can also interact with S3 programmatically using **Python's Boto3 library**.
- Example:**

```
import boto3
s3 = boto3.client('s3')
response = s3.list_buckets()

for bucket in response['Buckets']:
    print(bucket['Name'])
```

- **AWS S3 Bucket Permissions**

- You can manage **permissions** in Amazon S3 using several methods to control who can access your buckets and what actions they can perform.

- **1. Bucket Policies**

- Bucket policies are written in **JSON format** and attached directly to an S3 bucket.
- They define **who can access** the bucket and **what operations** they can perform (like upload, download, or delete).
- These policies apply at the **bucket level**.
- You can create or manage bucket policies using the **AWS Console, CLI, or Python SDK**.

- **2. Access Control Lists (ACLs)**
- **ACLs** are an older (legacy) method to manage access permissions.
- They allow you to grant **read** or **write** permissions to specific users or make objects **public**.
- However, AWS now **recommends using bucket policies** instead of ACLs for better control and flexibility
- **3. IAM Policies**
- **IAM (Identity and Access Management)** policies manage permissions for **users, groups, and roles** across AWS.
- You can attach IAM policies to allow specific users or roles to perform actions on S3 buckets.
- These policies provide **fine-grained control** and are ideal for managing permissions at the **account level**.

- **Features of Amazon S3**

- Amazon S3 offers several powerful features that make it one of the most reliable and scalable storage services in the cloud.
- **1. Durability**
 - Amazon S3 provides **99.999999999% (11 nines)** durability.
 - This means your data is **extremely safe** — the chance of losing it is almost zero.
- **2. Availability**
 - S3 ensures **99.99% uptime** for standard access.
 - **Availability** means you can always access your data, while **durability** ensures your data isn't lost.

- **3. Server-Side Encryption (SSE)**
- Amazon S3 provides built-in encryption options for securing data:
- **SSE-S3:** AWS manages the encryption keys.
- **SSE-C:** You manage your own encryption keys.
- **SSE-KMS:** AWS Key Management Service (KMS) handles key management securely.
- **4. File Size Support**
- You can store files from **0 bytes up to 5 TB** in size.
- This range supports both small and very large datasets.
- **5. Infinite Storage Space**
- Amazon S3 is **infinitely scalable**, meaning you can store **any amount of data** as your needs grow.
- **6. Pay-As-You-Use**
- You only pay for the **storage and services you use**.
- There are no upfront costs — it's a **cost-effective, usage-based model**.

- **Advantages of Amazon S3**

- Amazon S3 offers several key benefits that make it a popular and reliable cloud storage service.

- **1. Scalability**

- Amazon S3 can **automatically scale** to handle any amount of data.

- It grows **horizontally** without manual setup or human intervention.

- **2. High Availability**

- S3 provides **99.9% uptime**, ensuring your data is **always accessible**.

- You can access your files **from any region at any time**.

- **3. Data Lifecycle Management**

- You can **automate data movement** and deletion using **lifecycle rules**.

- For example, old data can automatically move to **Glacier or Standard-IA** for lower-cost storage.

- **4. Integration with Other AWS Services**

- S3 integrates seamlessly with other AWS services like **Lambda**, **CloudFront**, and **Athena**.

- For instance, a **Lambda function** can be triggered when a new file is uploaded to an S3 bucket.